KRE8ΩR

GETTING STARTED GUIDE

Timeline · Hardware · Claude Code · Zero-Mistake Setup

---

7 Kin Homestead · March 2026 · Read this before opening Claude Code

WHAT THIS DOCUMENT IS

This is your practical onboarding guide. It covers three things: how long this realistically takes to build, what hardware to gather now, and a step-by-step zero-mistakes guide to opening your first Claude Code project and getting it running. Read it end to end before doing anything.

PART 1 — REALISTIC TIMELINE

The Honest Answer

With no financial constraints on API and credits, and using Claude Code as your primary build environment, here is a realistic timeline to a fully functional personal beta — meaning you can run a real video through the complete pipeline from idea to distribution with minimal manual steps.

> **BASELINE ASSUMPTION**
> You are working on this consistently but not full-time. Roughly 2-4 focused hours per week, using Claude Code sessions of 1-2 hours each. If you go harder — daily sessions, treating it like a job — cut these timelines roughly in half.

Phase	What Gets Built	Duration	Sessions	Status	What You Can Do After
Done	**M1-M4 Prototype Tools**	Complete	Done	Done	Run any video through all 4 tools manually
Phase 1	**Unified App + Database**	3-4 weeks	6-8	Next	All tools talk to each other, no copy-paste between them
Phase 2	**EditΩr (VaultΩr + CutΩr)**	4-6 weeks	8-12	Soon	Drag footage in, get clips out automatically
Phase 2b	**AssemblΩr + SoundΩr**	3-4 weeks	6-8	Soon	Rough cut assembled automatically with music
Phase 3	**Id8Ωr + DirectΩr**	4-5 weeks	8-10	Later	Script to shoot plan, shoot day dashboard live
Phase 4	**M5 Automation Layer**	3-4 weeks	6-8	Later	Full pipeline fires automatically on YouTube upload
Phase 5	**OperatΩr + Intelligence**	2-3 weeks	4-6	Later	System learns from performance, suggests next topics
BETA	**Full System Integration Test**	2 weeks	4-5	Goal	Complete video cycle: idea to distribution, hands-off

Total Timeline: 5-6 Months to Personal Beta

That is the realistic number for consistent part-time effort. Here is what that actually looks like broken down:

- **Weeks 1-4: Phase 1. The least glamorous phase but the most important. Getting the foundation right means everything else builds cleanly. You will see the four tools connect and data flow between them for the first time. That moment will feel significant.**
- Weeks 5-14: Phase 2. This is where the system starts feeling like a real production tool. Footage goes in, clips come out, rough cuts assemble. You will probably start using VaultΩr as a daily tool before Phase 2 is even finished.

- Weeks 15-24: Phase 3. The upstream pipeline. Scripts, shot lists, shoot day management. By this point the downstream (distribution) is already running smoothly, so Phase 3 integrates into a system that already works.
- Weeks 25-28: Phase 4 automation. The pipes connect. This is the phase where the system stops being a tool you use and starts being a system that runs. The first time a YouTube upload fires the whole pipeline automatically will be a landmark moment.
- Weeks 29-32: Phase 5 and integration testing. The system learns. You run real videos through it, find the rough edges, tune the prompts, tune the voice, make it yours.

> **THE ACCELERATOR FACTOR**
> Every phase you complete makes the next phase faster. The database schema is already designed. The voice profile is already written. The integration patterns repeat. Claude Code sessions get more efficient as the codebase grows because Claude has more context to work from. The back half of this build will move faster than the front half.

What You Can Use Right Now While Building

You do not have to wait for beta to get value. The prototype tools are live and working today:

- M2 is running. Generate content packages for every video you make from this week forward.
- M3 is running. Generate captions for all 5 platforms per clip. Stop doing this manually.
- M4 is running. Generate your email sequence for every video. Stop skipping emails.
- M1 is running. Use the approval dashboard to manage your pipeline visually.

The manual copy-paste between tools is temporary. It disappears in Phase 1. But the tools themselves are real and usable now. Start using them. Every video you run through them trains your eye for what to tune when Phase 1 connects them.

PART 2 — HARDWARE

The Short Answer

Start with what you have. Upgrade strategically. The system is not hardware-intensive because most of the real work happens in the cloud via APIs. Your local machine is the coordinator, not the processor.

There is one exception: the EditΩr phase, specifically Whisper transcription and ffmpeg video processing. These are CPU-bound tasks that run locally. For everything else, a mid-range machine from 2018 onwards is fine.

Component	Minimum	Recommended	Why It Matters
RAM	8 GB	16 GB	Playwright runs a full Chrome instance. Whisper + ffmpeg running simultaneously needs headroom. 8GB works, 16GB is comfortable.
CPU	Intel i5 / Ryzen 5 (2018+)	Intel i7 / Ryzen 7 (2020+)	Whisper transcription is CPU-bound. A faster CPU means a 20-minute video transcribes in 3 min vs 8 min. Not a blocker, just a comfort thing.
Storage	100 GB free	1 TB SSD dedicated	Raw video footage adds up extremely fast. 4K footage from a single shoot day can be 50-100 GB. Plan for this early.
GPU (optional)	Not required	Any NVIDIA with CUDA	Whisper runs significantly faster on a CUDA-enabled GPU. Not required but cuts transcription time by 5-10x if available.
OS	Windows 10+, macOS 11+, Ubuntu 20+	macOS or Ubuntu	All fully supported. macOS and Ubuntu have slightly smoother Node.js and ffmpeg experiences but Windows works fine.
Internet	25 Mbps upload	100 Mbps+ upload	Uploading multiple video clips to multiple platforms simultaneously. Slow upload is the real bottleneck.
Uptime	On when agent runs	Dedicated always-on machine	The automation layer needs the machine awake. A dedicated device solves this permanently.

The Three Hardware Paths

Path A — Start Now (Free)

Use whatever laptop or desktop you have right now. If it meets the minimum specs above, it runs the system. Phases 1 and 2 will work fine on almost any modern machine. The only potential pain point is Whisper transcription speed on older hardware, but it still works — just takes a few extra minutes.

> **DO THIS NOW**

> Open your current machine, check available RAM and free storage. If you have 8GB RAM and 100GB free, you are good to start. Clear space if needed.

Path B — The Smart Upgrade ($100-200, eBay)

A used Mac Mini (2018 or later) or a refurbished Intel NUC is the single best hardware investment for this system. Here is why:

- Sits on a shelf. Never closes. Never sleeps. Always available when the automation layer needs it.
- Uses 10-15 watts. Costs roughly $1-2/month in electricity. Cheaper than a cloud server for light use.
- Mac Mini 2018 with 8GB RAM runs everything in the stack comfortably. 16GB model is better but costs more.
- Intel NUC i5 or i7 models are slightly faster for Whisper transcription and widely available on eBay for $80-150.

You do not need this on day one. Get the system running on your current machine first, prove it works, then move it to a dedicated device when you are ready to run it 24/7.

> **WHAT TO SEARCH ON EBAY**
> "Mac Mini 2018" or "Intel NUC i5 2019" or "Intel NUC i7 2020". Filter: Buy It Now, Good condition or better. Budget: $100-180. This is the recommended upgrade path.

Path C — Cloud Server ($6-12/month, when ready)

A DigitalOcean Droplet or equivalent removes the local machine dependency entirely. The automation layer runs in the cloud, always on, always connected, no hardware to maintain. This is the long-term home for the system once it is stable.

- Start with a $6/month Basic Droplet (1 vCPU, 1GB RAM) for the coordination layer
- Upgrade to $12/month (2 vCPU, 2GB RAM) when Whisper and ffmpeg processing moves to the server
- This is Phase 4 territory — do not set this up until the system is proven locally

Total infrastructure cost at full operation: approximately $12-18/month for the server, $2-5/month for Claude API at your video output volume. The whole system costs less than one Opus Clip session per month.

Storage Strategy — Plan This Now

Raw video footage is the one area where you need to think ahead. A shoot day of 4K footage can be 50-100GB. Over a year of content that is 2-5TB of raw footage. Plan your storage architecture before VaultΩr goes live:

- Working drive: fast internal SSD for active projects and the application. Minimum 500GB, ideally 1TB.
- Footage archive: large external HDD for the VaultΩr database of all past footage. A 4TB or 8TB drive is appropriate. Prices: $80-120 for a quality 4TB, $120-180 for 8TB.
- Backup: a second matching drive, or a NAS (Network Attached Storage) device, for automated backup. Data loss here means losing your entire footage library.
- Cloud backup: Backblaze B2 at $6/TB/month is the recommended cloud backup for footage. Non-negotiable once VaultΩr has real content in it.

START SHOPPING NOW

A 4TB external HDD and a backup solution should be on your list before VaultΩr goes live. Western Digital Elements or Seagate Backup Plus are reliable choices at $80-120. Do not wait until the footage database is full to think about this.

PART 3 — CLAUDE CODE: ZERO-MISTAKE SETUP

What Claude Code Is

Claude Code is a command-line tool that lets Claude read, write, and edit files directly on your computer. Instead of copy-pasting code back and forth in a chat window, Claude Code can see your entire project, write files directly, run commands, install packages, and build the system incrementally across many sessions.

It is significantly more powerful than a chat conversation for building software. It is also where mistakes can have real consequences — Claude is touching real files on your real computer. This section makes sure you do it right the first time.

> ⚠ **IMPORTANT**
> Claude Code is a powerful agentic tool. It will write files, run commands, and make changes to your system. The instructions below include several safety checkpoints. Do not skip them.

Prerequisites — Do These First

Before you open Claude Code, you need three things installed on your machine. Each one has a step-by-step process below.

Step 1: Install Node.js

Node.js is the runtime environment that runs the Kre8Ωr application. Claude Code also requires it.

1. Go to nodejs.org in your browser
2. Click the button that says LTS (Long Term Support) — do NOT click Current
3. Download the installer for your operating system (Windows .msi, macOS .pkg)
4. Run the installer. Click through all the defaults. Do not change anything.
5. When installation finishes, open Terminal (Mac) or Command Prompt (Windows)
6. Type: node --version and press Enter
7. You should see something like v20.11.0. If you see a version number, Node.js is installed correctly.

> ✓ **CHECKPOINT**
> You should see a version number starting with v18 or higher. If you see ‘command not found’ or ‘not recognized’, restart your computer and try again.

Step 2: Install Claude Code

Claude Code is installed as a Node.js package from the command line.

8. Open Terminal (Mac: press Cmd+Space, type Terminal, press Enter) or Command Prompt (Windows: press Windows key, type cmd, press Enter)
9. Type the following exactly and press Enter:

```
npm install -g @anthropic-ai/claude-code
```

10. Wait for installation to complete. You will see a lot of text scroll by. This is normal.

11. When it finishes, type: claude --version
12. You should see a version number. Claude Code is installed.

> ⚠ **IMPORTANT**
> If you see a permissions error on Mac, type: sudo npm install -g @anthropic-ai/claude-code and enter your computer password when prompted. Your password will not show as you type. This is normal.

Step 3: Get Your Anthropic API Key (Safely This Time)

> 🔴 **STOP AND READ THIS**
> Do NOT paste your API key into any chat window, document, email, or text message. Your API key is a password. It goes in one place: the Claude Code setup screen. Nowhere else.

13. Go to console.anthropic.com in your browser
14. Sign in to your Anthropic account
15. Click API Keys in the left sidebar
16. If you have any old keys from this conversation, click Disable on each one now
17. Click Create Key
18. Give it a name: Kre8r-Claude-Code
19. Copy the key. It starts with sk-ant-
20. Open a plain text file (Notepad on Windows, TextEdit on Mac) — NOT a Word document, NOT an email
21. Paste the key there temporarily while you complete setup
22. You will paste it into Claude Code in the next section. After that, delete this text file.

> ✓ **CHECKPOINT**
> The key is now safely stored in a local text file for the next 10 minutes. Do not close that file yet.

Creating Your Project Folder

Before opening Claude Code, create the folder where your Kre8Ωr project will live. This is important to do before starting Claude Code so you know exactly where everything goes.

On Mac:

23. Open Finder
24. Go to your home folder (the one with your name)
25. Create a new folder called: kre8r
26. Inside kre8r, create a folder called: docs
27. Copy all of the following files into the docs folder:

- Kre8r-Master-System-Bible-v1.0.docx
- kre8r-approval-dashboard.html
- kre8r-m2-package-generator.html
- kre8r-m3-caption-generator.html
- kre8r-m4-email-generator.html
- All five original 7-Kin docx files from your Google Drive

On Windows:

28. Open File Explorer
29. Go to C:\Users\YourName\
30. Create a new folder called: kre8r
31. Inside kre8r, create a folder called: docs
32. Copy all the files listed above into the docs folder

> ✓ **CHECKPOINT**
> Your project folder is ready. You have: C:\Users\YourName\kre8r\ (Windows) or /Users/YourName/kre8r/ (Mac) with a docs subfolder containing all reference documents.

Opening Claude Code for the First Time

Now you are ready. Follow these steps exactly, in order, without skipping.

Step 1: Navigate to Your Project Folder

Open Terminal (Mac) or Command Prompt (Windows) and navigate to your project folder:

On Mac, type:

```
cd ~/kre8r
```

On Windows, type:

```
cd C:\Users\YourName\kre8r
```

(Replace YourName with your actual Windows username)

Press Enter. The prompt should now show your kre8r folder.

Step 2: Start Claude Code

Type the following and press Enter:

```
claude
```

Claude Code will start. The first time it runs, it will ask for your API key.

Step 3: Enter Your API Key

When Claude Code asks for your API key:

33. Go to the plain text file where you temporarily saved your key
34. Copy the key (Ctrl+C on Windows, Cmd+C on Mac)
35. Paste it into the Claude Code prompt (Ctrl+V or Cmd+V)
36. Press Enter
37. Go back to that plain text file and delete it now
38. Empty your trash

> **✓ CHECKPOINT**
> Your API key is now stored securely by Claude Code. You will not need to enter it again on this machine.

Step 4: Read the Safety Prompt

Claude Code will show you information about what it can do and ask you to confirm. Read it. Then type yes and press Enter to proceed.

> **⚠ IMPORTANT**
> Claude Code will ask for permission before making significant changes. When it asks, read what it is about to do before saying yes. You are in control at all times. If something does not look right, type no and ask it to explain.

Step 5: Give Claude Code the Opening Brief

Once Claude Code is running and ready (you will see a > prompt), paste in the following opening message. This is the exact prompt from Section 15 of the System Bible:

> **COPY AND PASTE THIS EXACTLY**

> I am building Kre8Ωr — a complete AI-native content production OS for solo creators. The Master System Bible is in the docs/ folder of this project. Four prototype tools are also in docs/ (the four HTML files). Your first job is to: (1) read the bible completely, (2) initialize a Node.js project with a proper package.json and project structure, (3) create creator-profile.json for the 7 Kin Homestead instance using the voice and community details in the bible, (4) set up a SQLite database with the schema described in Section 11, (5) set up an Express.js local server that serves all four prototype tools connected through the shared database. Ask me nothing about intent — it is all in the bible. Ask me only about credentials and local file paths.

Press Enter. Claude Code will start reading the documents and planning the build. This first session may take 15-30 minutes. Let it work. Watch what it builds.

> ⚠ **IMPORTANT**
> Claude Code will ask you for permission before creating files, running npm install, or making significant changes. Always read what it is about to do. Type yes to approve, no to pause and discuss.

What to Expect in Your First Session

Here is what a healthy first Claude Code session looks like so you know you are on track:

- Claude Code reads the System Bible and confirms it understands the project structure
- It creates a package.json file — this is the project manifest. Normal.
- It runs npm install to set up dependencies — you will see a lot of text. Normal.
- It creates the folder structure: /src, /public, /database, /config
- It creates creator-profile.json with 7 Kin Homestead details
- It creates the SQLite database schema
- It sets up a basic Express.js server
- It asks you to run: node src/server.js
- You run that command in a new Terminal window
- You open http://localhost:3000 in your browser
- You see the beginning of the unified Kre8Ωr application

> **✓ CHECKPOINT**
> If you can open http://localhost:3000 in your browser and see a Kre8Ωr interface, Phase 1 is underway. That is a successful first session.

Session Management — How to Work With Claude Code Across Multiple Sessions

Claude Code does not remember previous sessions automatically. Each time you start a new session, you need to reorient it. Here is the pattern that works best:

Starting a New Session

39. Open Terminal, navigate to your kre8r folder: cd ~/kre8r
40. Start Claude Code: claude
41. Paste this at the start of every session:

> **SESSION RESTART PROMPT**
> I am continuing the Kre8Ωr build. Read the System Bible in docs/ and review the current state of the codebase to understand where we left off. Then tell me: what is the current state of the project, what was last completed, and what is the logical next step according to the build order in Section 12?

42. Claude Code will review the codebase and tell you where things stand
43. Confirm its understanding, then give it the next task

Ending a Session Cleanly

Before ending any Claude Code session, ask it to do this:

> **END OF SESSION PROMPT**
> Before we end this session: (1) summarize what was built or changed today in a file called SESSION-LOG.md in the project root, (2) update a TODO.md file with the next 3 specific tasks for the next session, (3) make sure all changes are saved.

This creates a paper trail. Future sessions can read SESSION-LOG.md and TODO.md to pick up exactly where you left off without confusion.

Safety Rules for Claude Code Sessions

These rules protect you from the most common mistakes people make when using agentic coding tools:

🔴 **STOP AND READ THIS**

Never paste API keys, passwords, or credentials into the Claude Code chat window. Claude Code has a secure credential store. Use it. If Claude Code asks for a credential, it will tell you the safe way to provide it.

⚠ **IMPORTANT**

Never say yes to a Claude Code action you do not understand. If it says it is about to delete files, modify system settings, or install something unfamiliar, type no and ask it to explain first.

BACK UP BEFORE BIG SESSIONS

Before any session where you are making major changes (new phases, database migrations, integration work), copy your entire kre8r folder to a backup location. This takes 30 seconds and has saved many projects.

GIT FROM DAY ONE

Ask Claude Code to initialize a Git repository in your first session. Git is version control — it tracks every change and lets you undo anything. You do not need to understand Git deeply. Just let Claude Code set it up and commit changes at the end of each session.

Troubleshooting — Common First-Session Problems

"claude" is not recognized as a command

Close Terminal and reopen it. If still not working, try: npx @anthropic-ai/claude-code. If that works, the installation path is not set correctly — ask Claude Code to help you fix it in your next attempt.

Permission denied errors on Mac

Prefix the install command with sudo: sudo npm install -g @anthropic-ai/claude-code. Enter your computer password when prompted.

Claude Code is asking for my API key again

Your key may not have saved correctly. Re-enter it. This time it should stick.

Claude Code made a change I did not want

Type: undo the last change. Claude Code can reverse most of its own actions. If the change involved deleting files, check your Trash before emptying it.

The server is not starting (node src/server.js fails)

Paste the error message into Claude Code and say: fix this error. It will diagnose and resolve it.

I closed Terminal and now I cannot get back to the project

Open a new Terminal, type cd ~/kre8r (Mac) or cd C:\Users\YourName\kre8r (Windows), then type claude to restart Claude Code.

PART 4 — YOUR FIRST 30 DAYS

Week 1 — Foundation

- Day 1: Complete Parts 1-3 of this guide. Get Node.js installed, Claude Code running, first session complete.
- Day 2-3: Review what Claude Code built. Open the app in your browser. Does it look right? Run M2 through the unified app on a real video.
- Day 4-7: Session 2 with Claude Code. Goal: all four prototype tools connected through the database. Copying between tools should start disappearing.

Week 2 — Stabilize and Use

- Run every video you make this week through M2, M3, M4. Use the tools. Find what feels off.
- Note every friction point: what still requires manual steps, what captions miss the voice, what emails feel generic.
- Session 3: share your friction notes with Claude Code. Tune the prompts. Fix the rough edges.
- Do not rush to Phase 2 until Phase 1 feels solid.

Week 3 — VaultΩr Foundation

- Session 4: begin VaultΩr. The drag-and-drop footage intake, ffprobe metadata extraction, folder organization.
- Test it on one real shoot day of footage. Watch how it categorizes. Fix what it gets wrong.
- This is the phase where the system starts feeling fundamentally different from any tool you have used before.

Week 4 — Review and Plan

- Do not add new features. Review everything built so far.
- Run a full manual pipeline: M2 → M3 → M4 → M1 on a real video. Is data flowing? What still requires copy-paste?
- Update your TODO.md with the specific gaps. Make a plan for Month 2.
- Celebrate what is working. The foundation of a serious piece of software is live on your machine.

> **THE MONTH 1 GOAL**
> By day 30, you should have: a unified local web app serving all four prototype tools, a SQLite database tracking pipeline state, the beginning of VaultΩr, and zero copy-paste required between M2, M3, M4, and M1. That is a genuinely significant piece of software. Do not underestimate it.

APPENDIX — QUICK REFERENCE COMMANDS

Terminal Commands You Will Use Constantly

Navigation

```
cd ~/kre8r                      # Go to project folder (Mac)
cd C:\Users\YourName\kre8r     # Go to project folder (Windows)
ls                              # List files in current folder (Mac)
dir                             # List files in current folder (Windows)
pwd                             # Show current folder path (Mac)
```

Claude Code

```
claude                          # Start Claude Code in current folder
claude --version                # Check Claude Code version
```

Node.js / Project

```
node --version                  # Check Node.js version
npm install                     # Install project dependencies
node src/server.js              # Start the Kre8r local server
Ctrl+C                          # Stop the server
```

Git (once Claude Code sets it up)

```
git status                      # See what has changed
git add .                       # Stage all changes
git commit -m "session notes" # Save a snapshot
git log --oneline               # See history of changes
```

URLs You Will Use

- http://localhost:3000 — your local Kre8Ωr app once the server is running
- console.anthropic.com — your Anthropic account, API keys, usage
- nodejs.org — Node.js downloads
- digitalocean.com — cloud server when ready for Phase 4
- bitwarden.com — password manager for all API keys (already in your stack)